

# אבטחה בסיסית

## איך מתבצע Authentication

Supabase Auth, דוא"ל+סיסמה בלבד — ללא magic links, ללא OAuth, ללא הרשמה עצמית פומבית (`enable_signup = false` ב-`supabase/config.toml`, והרשמה עצמית מבוטלת גם ברמת פרויקט Supabase). כל חשבון נוצר על ידי המנהל: המנהל יוצר עובד/ת ישירות דרך `/admin/personnel` (`createWorkerAccount`), באמצעות Admin API עם מפתח `service-role` — ראו בהמשך), או, עבור חשבון המנהל היחיד של האפליקציה, דרך סקריפט בוטסטרפ חד-פעמי המתואר ב-README, מכיוון שאין במכון נתיב ממשק באפליקציה שיכול ליצור חשבון בתפקיד `admin`.

`session`-ים הם עוגיות `httpOnly` המנוהלות על ידי `lib/supabase/server.ts` (`@supabase/ssr`) ו-`lib/supabase/client.ts` — לעולם לא נגישות ל-JavaScript בצד לקוח. `proxy.ts` קורא ל-`supabase.auth.getClaims()` בכל בקשה דרך `updateSession` של `lib/supabase/proxy.ts`, וזה מה שבאמת מרענן טוקן `session` שעומד לפוג; זה ממוקד במכון לרענון `session` בלבד, לא לשמירה על תפקיד (ראו בהמשך). סיסמאות דורשות מינימום 8 תווים, הועלה מברירת המחדל של Supabase שהיא 6 (`minimum_password_length = 8`).

## איך מתבצע Authorization

שתי שכבות עצמאיות, במכון יתירות — זו החלטת האבטחה המרכזית של האפליקציה, ונבדקת גם ביחידה וגם באינטגרציה (§4 `docs/test-spec.md`):

1. **שומר ברמת האפליקציה** — `lib/auth.ts` (`requireAdmin()` / `requireWorker()`) בודק את `profiles.role` של הקורא ומפנות (`redirect()`) מכל עמוד או Server Action שתפקיד שגוי מגיע אליו. זה מה שגורם לממשק להתנהג נכון (עמוד מנהל אף פעם לא מרונדר לעובד), אבל לפי תיעוד האבטחה של Next.js עצמה, סינון ברמת רינדור אינו גבול אבטחה, מכיוון שבקשות יכולות להישלח בלי לעבור דרך הממשק — לכן לא מספיק בפני עצמו. מכאן שכבה 2.
2. **Row-Level Security (Postgres)** — הגבול האמיתי הנאסף, מתחת לכל טבלה. `is_admin()` (פונקציית SQL מסוג `security definer`, עם `search_path` מוצמד כדי להימנע ממלכודת ה-`injection` הקלאסית של `search_path` על פונקציות `security definer`) עומדת מאחורי רוב מדיניות הכתיבה (`for all` `using (is_admin()) with check (is_admin())`), וכל מדיניות ממוקדת ל-`auth.uid()` עבור נתונים אישיים. זו אכיפה אמיתית של Postgres, לא מוסכמה ברמת האפליקציה — היא חלה גם אם בקשה איכשהו עוקפת לגמרי את השומר ברמת האפליקציה.

הכלל היחיד ששווה לציין במפורש: **נראות assignments מבוטלת-תזמון, לא רק תפקיד**. עובד יכול לראות את השיבוץ שלו עצמו רק ברגע שה-`published_at` של המשמרת שלו מוגדר — אף פעם לא לפני כן, גם אם זו באופן חד-משמעי "השורה שלו":

```
create policy assignments_select on assignments
for select using (
  is_admin()
```

```
or (worker_id = auth.uid() and exists (
  select 1 from shifts s where s.id = assignments.shift_id and s.published_at
  is not null
))
);
```

זה אומת התנהגותית עם session-ים אמיתיים של חשבונות בדיקה מחוברים, לא רק על ידי קריאת ה-SQL – גם בשלב הפיתוח המקורי וגם שוב בחבילת האינטגרציה האוטומטית של הפרויקט (tests/integration/authorization.test.ts).

## אילו פעולות מותרות רק למשתמש מחובר

למעשה הכול. ל-anon (בקשה לא-מאומתת) יש אפס הרשאות על כל טבלה:

```
grant usage on schema public to authenticated;
grant select, insert, update, delete on all tables in schema public to
authenticated;
-- אין כלום – כל טבלה דורשת חיבור anon-ל
```

בשילוב עם "חשיפה אוטומטית של טבלאות חדשות" כביה ברמת פרויקט Supabase (טבלאות חדשות מקבלות אפס הרשאות API כברירת מחדל, לא התנהגות ה-auto-expose הישנה), טבלה נגישה דרך ה-API רק ברגע שיש לה גם GRANT מפורש וגם מדיניות RLS – שני צעדים מכוונים, לא ברירת מחדל בטעות. העמודים היחידים הנגישים ללא חיבור הם /login ועמודי השגיאה/לא-נמצא של ברירת המחדל של Next.js.

## איך מונעים גישה למידע של משתמש אחר

RLS שוב – כל טבלת "נתונים אישיים" (availability, worker\_qualifications, notifications, assignments) ממקדת את מדיניות ה-select/insert/update שלה ל-worker\_id = auth.uid() (או is\_admin()), נאכף ב-Postgres ללא תלות במה שקוד האפליקציה עושה או שוכח לעשות. אומת ישירות, לא הונח: tests/integration/authorization.test.ts מתחבר כשני חשבונות עובד זמניים אמיתיים נפרדים ובודק שאחד לא יכול לקרוא את שורות worker\_qualifications של השני (השאלתה מחזירה אפס שורות, לא שגיאה – RLS מסננת בשקט, לא דוחה).

דיווח עצמי של עובד הוא המקום היחיד שעובד כתב שורה על עצמו שמנהל יפעל לפיה בהמשך, וסעיף ה-with check הוא מה שמונע ניצול לרעה כנתיב הסלמת הרשאות:

```
create policy worker_qualifications_self_report on worker_qualifications
for insert with check (
  worker_id = auth.uid() and source = 'self_reported' and status = 'pending'
);
```

עובד לא יכול להכניס שורה שטוענת status = 'approved' או source = 'admin\_granted' עבור עצמו – מסד הנתונים דוחה זאת, ללא תלות במה שקוד הטופס של הדיווח העצמי עצמו מאמת או לא מאמת.

## איך מבצעים ולידציה לקליטים

שוב שתי שכבות, מאותה סיבה כמו ה-authorization — הגנה בשכבות, לא מאמץ כפול:

1. **שכבת האפליקציה** — כל Server Action שמקבלת קלט טופס מאמתת אותו עם סכמת Zod לפני נגיעה במסד הנתונים (features/\*/actions.ts), כולל כללים חוצי-שדות (shiftSchema 's) סוף-אחרי-התחלה, windowSchema 's סגירה-אחרי-פתיחה). פענוח שנכשל מחזיר שגיאה בעברית ידידותית ולעולם לא מגיע לשאלתה. נבדק ביחידה ישירות עבור ארבע סכמות מייצגות על פני צורות הוולידציה השונות באפליקציה — docs/test-spec.md §2.
2. **שכבת מסד הנתונים** — כמה כללים שחשובים לשלמות הנתונים ללא תלות באיזה נתיב קוד כותב את השורה נאכפים על ידי טריגרים של Postgres, לא רק קוד אפליקציה: גם worker\_qualifications וגם position\_qualifications אוכפים "option\_id" נדרש אם ורק אם להסמכה יש אפשרויות, וחייב להשתייך לאותה הסמכה" דרך טריגר, אומת ישירות מול הכנסות אמיתיות (תוך עקיפה מוחלטת של האפליקציה) לפני שנבנה עליו ממשק כלשהו. כללי ייחודיות (positions.name, qualifications.name), כלל "לכל" היותר הענקת הסמכה פעילה אחת" המבוסס-אינדקס-חלקי) נאכפים באותו אופן — אילוץ unique, לא מירוצ בדיקה-ואז-הכנסה ברמת האפליקציה.

## איך מגנים על קריאות ל-API

**אין שכבת REST/GraphQL API נפרדת** — לא קיים אף נתיב app/api/\* בקוד הזה. כל שינוי הוא Server Action של Next.js, שמקבלת כמה הגנות בחינם, לא כתובות ידנית (אומת ישירות מקריאת התייעוד המותקן בפועל של Next.js, לפי כלל הבית של הפרויקט הזה לבדוק את התייעוד המותקן לפני הסתמכות על התנהגות framework מידע אימון):

- **CSRF בדיקת** Next.js — **Origin** משווה את כותרת ה-Origin של הבקשה מול Host (או X-Forwarded-Host) ודוחה אי-התאמה, ללא צורך בקוד אפליקציה כדי להצטרף לזה.
- **מזהי action מוצפנים + הסרת קוד מת** — Server Action היא לא רק "פחות ניתנת לגילוי מ-endpoint REST", היא גם מוסרת באופן פעיל מה-bundle של הלקוח אם אינה בשימוש, וההפניה אליה מוצפנת בזמן build.

בנוסף לכך, כל Server Action עצמה מתחילה עם requireAdmin() / requireWorker() לפני כל דבר אחר, ולכל טבלה שהיא נוגעת בה יש RLS מתחת בכל מקרה. גישת קריאה (Server Components ששולפות נתונים) עוברת דרך אותו נתיב RLS / createClient() — אין "API קריאה" נפרד עם בדיקות חלשות יותר מנתיב השינוי.

שכבת PostgREST של Supabase עצמה, שאיתה לקוח ה-JS של Supabase מדבר מתחת למכסה, מוסיפה רשת ביטחון ברמת הפלטפורמה, בלתי תלויה בכל דבר בקוד האפליקציה הזו: max\_rows = 1000 (supabase/config.toml) מגביל כל תגובה בודדת ללא תלות בצורת השאלתה.

## איך שומרים סודות כמו מפתחות API

שלושה משתני סביבה, בשתי רמות ארון שונות מאוד:

- `NEXT_PUBLIC_SUPABASE_URL` / `NEXT_PUBLIC_SUPABASE_ANON_KEY` — מיועדים להיות ציבוריים (קידומת ה-`NEXT_PUBLIC` שולחת אותם לדפדפן בכוונה). בקרת גישה נאכפת על ידי RLS, לא על ידי שמירת המפתח הזה בסוד — זה המודל המתועד של Supabase עצמה, לא קיצור דרך שנלקח כאן.
  - `SUPABASE_SERVICE_ROLE_KEY` — עוקף RLS לגמרי, אז זה כן סוד אמיתי. אומת כבנוי-לצד-שרת-בלבד: הוא נקרא רק בקוד שרץ בשרת בלבד — `lib/supabase/admin.ts`
  - `features/accounts/actions.ts` (המסלול שהאפליקציה החיה עצמה משתמשת בו ליצירת חשבון עובד), ובנוסף שני סקריפטים חד-פעמיים שרצים דרך `node` מחוץ לאפליקציה עצמה (`scripts/create-` `scripts/seed-demo-data.mjs`, `admin.mjs`) — אף קובץ `"use client"` לא מפנה אליו, וללא קידומת `Next.js` `NEXT_PUBLIC` היה מסרב להטמיע אותו ב-`bundle` של לקוח גם בטעות.
- מקומית, שלושתם חיים ב-`.env.local`, שמתעלמים ממנו ב-`.env*` (`.gitignore`) עם חריגה מפורשת ל-`.env*.example`! עבור התבנית המחויבת) — לעולם לא מחויב, אומת על ידי כלל ההתעלמות ולא רק על ידי משמעת ידנית. בפרודקשן, אותם שלושה משתנים מוגדרים ישירות בהגדרות הפרויקט של Vercel, לא מחויבים בשום מקום ב-`repo` או בהיסטוריה שלו.

## אילו סיכוני אבטחה עדיין קיימים ומה הייתם משפרים בהמשך

פערים כנים, לא מוצגים כפתורים:

- **אין כאן שום עמידה בדרישות אבטחת מידע של גוף כמו טייסת בצה"ל, וזה לא נטען.** הפרויקט מתארח על תשתית אזרחית צד-שלישי (Supabase/Vercel) ונבנה מראש על נתונים סינתטיים בלבד, לא נתוני כוח אדם אמיתיים (ראו `docs/product-spec.md`) — בדיוק בגלל שאין כאן עמידה כזו. כל הניתוח במסמך הזה שייך לרמת הגנה מתאימה לפרויקט קורס על תשתית ציבורית. אימוץ אמיתי על ידי גוף כמו טייסת היה דורש התאמה משמעותית ומחדש-מהיסוד כדי לעמוד בדרישות הסיווג/אבטחת המידע הרלוונטיות (כולל, ברמת העיקרון, סוג התשתית עצמה — לא רק בקורות בתוך קוד האפליקציה) — לא תוספת קטנה מעבר לרשימה בהמשך.
- **אין הגבלת קצב (rate limiting) על `/login`** או פעולת הדיווח העצמי של עובד — ניסיון `brute-force` ממוסקרפט מול אימייל ידוע לא מוגבל היום על ידי קוד האפליקציה עצמה (ל-Supabase Auth יש הגבלה ברמת הפלטפורמה, אך שום דבר ספציפי לאפליקציה לא נוסף). היה נוסף לפני כל פריסה אמיתית (לא-כיתתית).
- **אין יומן ביקורת (audit log)** — פעולות מנהל (הענקת/ביטול הסמכה, פרסום לוח משמרות) לא נרשמות בשום מקום מעבר לשדות ה-`updated / reviewed_by` של השורה עצמה היכן שאלה קיימים במקרה. טבלת `audit` ייעודית שרק-מוסיפים אליה הייתה הצעד הטבעי הבא אם היה צריך לתמוך באחריותיות "מי עשה מה, מתי".
- **אין MFA** — דוא"ל+סיסמה בלבד, מתאים לסד"כ טייסת קטן וסגור עם העברת פרטי גישה מחוץ למערכת, אבל פער אמיתי אם סיכון פריצת חשבון היה גדל (למשל ארגון גדול משמעותית יותר, או הוספת איפוס סיסמה עצמאי בהמשך).
- **אין CSP או כותרות אבטחה אחרות מוגדרות** — ברירות המחדל של הפלטפורמה של Next.js/Vercel, חלות, אך שום דבר ספציפי לפרויקט (`Strict-Transport-Security`, `Content-Security-Policy` וכו') לא נוסף ב-`next.config.ts`. שווה להוסיף כצעד הקשחה זול גם בהיקף הזה.
- **ל-`getLatestRenewalDates` ולשאליות קריאה אחרות אין הגבלת עלות-שאלית מפורשת מעבר ל-`max_rows = 1000`** הפלטפורמתי של Supabase — לא פער בקרת-גישה (RLS עדיין חלה), אלא סיכון בצורת מניעת-שירות אם דפוסי הקריאה של האפליקציה הזו היו צריכים אי פעם לגדול הרבה יותר.
- **גם סקריפט הבוטסטרפ של המנהל וגם מפתח ה-`service_role` מייצגים "מפתחות לממלכה" בנקודת אמון יחידה** — מתאים לכלי עם מנהל יחיד וארגון יחיד, אבל היה דורש סבב-מפתחות אמיתי וחשבונות שירות בעלי-הרשאה-מינימלית לפני שהדפוס הזה יכול להתרחב לארגון או צוות מנהלים גדול יותר.